НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ім. Ігоря СІКОРСЬКОГО»
НАВЧАЛЬНО-НАУКОВИЙ ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

Звіт з виконання комп'ютерного практикуму №1
з дисципліни «Спеціальні розділи обчислювальної математики»

БАГАТОРОЗРЯДНА АРИФМЕТИКА

Виконав студент
групи ФІ-43
Деркач Трохим Тарасович

Київ — 2026

Зміст

1 Мета роботи **2**

2 Завдання до комп'ютерного практикуму **2**

3 Теоретичні відомості **2**

4 Результати роботи програми **2**

5 Висновки **3**

6 Додаток. Лістинг коду програми **5**

6.1 Файл bignumarifmetic.h 5
6.2 Файл bignumarifmetic.cpp 5
6.3 Файл tests.h 10
6.4 Файл tests.cpp 10
6.5 Файл benchmarks.h 13
6.6 Файл benchmarks.cpp 13
6.7 Файл main.cpp 15

1 Мета роботи

Отримання практичних навичок програмної реалізації багаторозрядної арифметики; ознайомлення з прийомами ефективної реалізації критичних по часу ділянок програмного коду та методами оцінки їх ефективності.

2 Завдання до комп'ютерного практикуму

Пункт А. Розробити бібліотеку функцій для роботи з m-бітними цілими числами, що підтримує числа довжиною до 2048 біт. Реалізувати операції:

- переведення малих констант у формат великого числа (зокрема, 0 та 1);
- додавання та віднімання чисел;
- множення чисел, піднесення чисел до квадрату;
- ділення чисел, знаходження остачі від ділення;
- піднесення числа до багаторозрядного степеня;
- перетворення числа в символьний рядок і назад (обов'язково — шістнадцяткове представлення, бажано — двійкове та десяткове);
- (бажано) визначення номера старшого ненульового біта та бітові зсуви.

Пункт Б. Проконтролювати коректність реалізації алгоритмів, зокрема перевірити тотожності $(a+b)\cdot c = c\cdot(a+b) = a\cdot c + b\cdot c$ та $n\cdot a = \underbrace{a+a+\cdots+a}_{n}$ для $n \geq 100$; розробити власні тести.

Пункт В. Обчислити середній час виконання реалізованих арифметичних операцій і кількість тактів процесора на кожну операцію; подати результати у вигляді таблиць.

3 Теоретичні відомості

Бібліотека працює з беззнаковими числами сталої довжини 2048 біт. Число подано в системі числення з основою 2^{32}: структура `BigNum` зберігає 64 цифри типу `uint32_t`, причому наймолодша цифра лежить у комірці з індексом 0 (little-endian). Проміжні значення (сума двох цифр із переносом, добуток двох цифр) обчислюються в 64-бітній змінній, тому переповнення не виникає. Результати, що перевищують 2048 біт, обрізаються, тобто всі операції виконуються за модулем 2^{2048}.

Додавання, віднімання та множення виконуються «в стовпчик». Для піднесення до квадрату кожен попарний добуток цифр $a_i a_j$ $(i \neq j)$ обчислюється один раз, а потім подвоюється. Ділення з остачею виконується побітово: дільник зсувається до старшого біта залишку і віднімається від нього, а в частці встановлюється відповідний біт. Степінь A^B обчислюється проходом по бітах показника B, починаючи зі старшого: на кожному кроці результат підноситься до квадрату і, якщо поточний біт дорівнює 1, множиться на A.

4 Результати роботи програми

Нижче наведено символьні зображення тестових чисел та результатів операцій над ними (пункт Б), а також середній час виконання операцій (пункт В).

```
--- Running Tests (2048 bit, all operations mod 2^2048) ---

  a = 123456789ABCDEF0FEDCBA9876543210
  b = F1E2D3C4B5A69788796A5B4C3D2E1F0
  c = 1234567890ABCDEF
```

```
  (a+b)*c   = 25E9D46E2ADA8EBB8E46D42F50C75802129F304F17FAC00
  a*c + b*c = 25E9D46E2ADA8EBB8E46D42F50C75802129F304F17FAC00
Test 1 [(a+b)*c == c*(a+b) == a*c + b*c]: Passed!
  100*a       = 71C71C71C71C716238E38E38E38E38E40
  a + ... + a = 71C71C71C71C716238E38E38E38E38E40
Test 2 [n*a == a+a+...+a, n = 100..1000]: Passed!
  a + b = 215283B4E61748698673604D3A271400
  a - b = 316293C4F627578774614E3B2815020
Test 3 [a+b == b+a, (a+b)-b == a, a-a == 0, carry/borrow]: Passed!
  a * b = 113366AB0066DE57C0175D92B6C9CBBB8A47F4901A93FD55EF9A562300EFF00
  a^2   = 14B66DC33F6ACDCCA2148A6A1A009454495D294750DF8CCDEEC6CD7A44A4100
Test 4 [a*b == b*a, a^2 == a*a, (a+b)^2 == a^2 + 2ab + b^2]: Passed!
  a^2 + 12345 = 14B66DC33F6ACDCCA2148A6A1A009454495D294750DF8CCDEEC6CD7A44A7139
  q = (a^2 + 12345) / b = 15EBD2CAD3EE192EC2C99D3DAAE4FCFD
  r = (a^2 + 12345) % b = 61F7E44949059EAB1A44063ECB8E709
Test 5 [a == b*q + r, 0 <= r < b, div by 0 -> error]: Passed!
  3^200 = 1FD5863C3EB0469EC21A937A76F3432FFD73D97E447606B683ECF6F6E4A7AE225BFAFF1EAAF8B0A1
Test 6 [a^(m+n) == a^m * a^n, a^(m*n) == (a^m)^n, a^0 == 1, a^1 == a]: Passed!
  a << 100 = 123456789ABCDEF0FEDCBA9876543210000000000000000000000000
  a >> 8   = 123456789ABCDEF0FEDCBA98765432
Test 7 [a << k == a*2^k, a >> k == a / 2^k, BitLength(2^k) == k+1]: Passed!
  ToBin(c) = 10010001101000101011001111000100100001010101111100110111101111
Test 8 [FromHex(ToHex(a)) == a, FromBin(ToBin(a)) == a, bad strings rejected]: Passed!

All tests passed (failed: 0)
------------------------------
```

```
--- Running Benchmarks (2048 bit, 1024 random numbers per operand) ---

| Operation                      | Iterations |      ns / op |  cycles / op |
|--------------------------------|------------|--------------|--------------|
| LongAdd                        |    1000000 |         22.7 |           57 |
| LongSub                        |    1000000 |         23.1 |           58 |
| LongCmp                        |    1000000 |          1.3 |            3 |
| BitLength                      |    1000000 |          8.7 |           22 |
| LongMulOneDigit                |    1000000 |         25.4 |           63 |
| LongShiftBitsToHigh            |    1000000 |         38.4 |           96 |
| LongShiftBitsToLow             |    1000000 |         44.1 |          110 |
| LongMul (128 bit)              |     200000 |        269.4 |          672 |
| LongMul (1024 bit)             |      50000 |       1908.9 |         4763 |
| LongMul (2048 bit)             |      50000 |       3823.9 |         9542 |
| LongSquare (128 bit)           |    1000000 |         96.9 |          242 |
| LongSquare (1024 bit)          |     200000 |        441.6 |         1102 |
| LongSquare (2048 bit)          |     200000 |        798.1 |         1991 |
| LongDivMod (2048 / 1024 bit)   |       2000 |      57187.2 |       142696 |
| LongDivMod (2048 / 32 bit)     |       1000 |     124933.9 |       311741 |
| LongPower (exponent 64 bit)    |       1000 |     149072.2 |       371972 |
| LongPower (exponent 2048 bit)  |         30 |    4683530.0 |     11686594 |
| ToHex                          |     200000 |        835.2 |         2084 |
| FromHex                        |     200000 |       1979.3 |         4939 |

cycles = TSC (__rdtsc), counts at the nominal CPU frequency
------------------------------
```

5 Висновки

У ході роботи розроблено бібліотеку багаторозрядної арифметики мовою C++ для беззнакових чисел фіксованої довжини 2048 біт (64 цифри по 32 біти, формат little endian). Реалізовано всі обов'язкові операції пункту A, а також бажані: визначення довжини числа, бітові та цифрові зсуви, двійкове представлення. Коректність підтверджено перевіркою тотожностей на тисячах випадкових наборів і порівнянням з незалежно обчисленими значеннями — усі тести пройдено.

Результати вимірювань узгоджуються з теоретичною складністю алгоритмів:

- **Додавання та віднімання** (≈ 23 нс, ≈ 57 тактів) мають складність $O(n)$: один прохід по 64 цифрах, менше одного такту на цифру. Множення на одну цифру має ту саму структуру і практично той самий час.

- **Порівняння** (≈ 1 нс) на випадкових числах завершується вже на старшій цифрі; у найгіршому випадку (рівні числа) воно також $O(n)$.
- **Множення** (≈ 3.8 мкс) має складність $O(n^2)$. Для кожної ненульової цифри множника виконуються три повні проходи по масиву (множення на цифру, зсув, додавання), тому час зростає пропорційно до кількості цифр множника: 128, 1024 та 2048 біт дають 0.27, 1.9 та 3.8 мкс.
- **Піднесення до квадрату** у ≈ 4.8 раза швидше за множення тих самих чисел. Це результат обчислення лише половини попарних добутків, накопичення результату без проміжних масивів і пропуску добутків, що виходять за межі 2048 біт.
- **Ділення** — найповільніша з базових операцій: алгоритм «зсувай і віднімай» визначає частку по одному біту, виконуючи на кожен біт зсув, порівняння, віднімання та обчислення довжини (кожне $O(n)$). Тому час пропорційний довжині частки: ділення на 32-бітне число (частка ≈ 2016 біт) повільніше за ділення на 1024-бітне (частка ≈ 1024 біти) у ≈ 2.2 раза.
- **Піднесення до степеня** має складність $O(m \cdot n^2)$, де m — кількість біт показника: на кожен біт виконується одне піднесення до квадрату і в середньому половина множення. Час для показників 64 та 2048 біт (0.15 та 4.7 мс) відрізняється в ≈ 31 раз, що відповідає співвідношенню довжин показників ($2048/64 = 32$).
- **Перетворення в рядок і назад** мають лінійну складність і завдяки основам-степеням двійки не потребують ділення.

Найбільший потенціал для подальшого прискорення має множення: перехід до накопичення добутків $a_j b_i$ одразу в позиції $i + j$ (як у піднесенні до квадрату) прибрав би проміжні проходи по масиву, а для довгих чисел ефективним стає алгоритм Карацуби.

6 Додаток. Лістинг коду програми

Повний код і історія змін доступні в репозиторії: https://github.com/troshxa/SROM_Derkach.

6.1 Файл bignumarifmetic.h

```h
#pragma once
#include <cstdint>
#include <string>

constexpr int W = 32;
constexpr int BITS = 2048;
constexpr int WORDS = BITS / W; //w=32
constexpr uint64_t BASE = 1ULL << W;
constexpr uint64_t MASK = BASE -1;

struct BigNum {
    uint32_t d[WORDS];
};

uint32_t LongAdd(const BigNum& a, const BigNum& b, BigNum& c);
uint32_t LongSub(const BigNum& a, const BigNum& b, BigNum& c);
int LongCmp(const BigNum& a, const BigNum& b);
BigNum ToBigNum(uint64_t x);
uint32_t LongMulOneDigit(const BigNum& a, uint32_t b, BigNum& c);
BigNum LongShiftDigitsToHigh(const BigNum& a, int x);
BigNum LongShiftDigitsToLow(const BigNum& a, int x);
BigNum LongShiftBitsToHigh(const BigNum& a, int x);
BigNum LongShiftBitsToLow(const BigNum& a, int x);
void LongMul(const BigNum& a, const BigNum& b, BigNum& c);
void LongSquare(const BigNum& a, BigNum& b);
int BitLength(const BigNum& a);
bool LongDivMod(const BigNum& a, const BigNum& b, BigNum& q, BigNum& r);
void LongPower(const BigNum& a, const BigNum& b, BigNum& c);
bool FromHex(const std::string& s, BigNum& a);
std::string ToHex(const BigNum& a);
bool FromBin(const std::string& s, BigNum& a);
std::string ToBin(const BigNum& a);
```

6.2 Файл bignumarifmetic.cpp

```cpp
#include "bignumarifmetic.h"

uint32_t LongAdd(const BigNum& a, const BigNum& b, BigNum& c){
    uint32_t carry = 0;
    uint64_t temp = 0;
    for (int i = 0; i < WORDS; i++){
        temp = (uint64_t)a.d[i] +b.d[i] + carry;
        c.d[i] = temp & MASK;
        carry = temp >> W;
    }
    return carry;
}

uint32_t LongSub(const BigNum& a, const BigNum& b, BigNum& c){
    uint32_t borrow = 0;
```

```
    int64_t temp = 0;
    for(int i = 0; i <WORDS; i++){
        temp = (int64_t)a.d[i] - b.d[i] - borrow;
        if(temp>=0){
            c.d[i] = (uint32_t)temp;
            borrow = 0;
        }else{
            c.d[i] = (uint32_t)(BASE + temp);
            borrow = 1;
        }
    }
    return borrow;
}

int LongCmp(const BigNum& a, const BigNum& b){
    int i = WORDS-1;
    while(i >=0 && a.d[i] == b.d[i]){
        i-=1;
    }
    if(i==-1) return 0;
    else{
        if(a.d[i]>b.d[i]) return 1;
        else return -1;
    }

}

BigNum ToBigNum(uint64_t x){
    BigNum a ={};
    a.d[0] = x & MASK;
    a.d[1] = x >> W;
    return a;
}

uint32_t LongMulOneDigit(const BigNum& a, uint32_t b, BigNum& c){
    uint32_t carry =0;
    uint64_t temp = 0;
    for(int i =0; i < WORDS; i++){
        temp = (uint64_t)a.d[i]*b + carry;
        c.d[i] = temp & MASK;
        carry = temp >> W;
    }
    return carry;
}

BigNum LongShiftDigitsToHigh(const BigNum& a, int x){
    BigNum c = {};
    if (x >= WORDS) return c;
    for(int i = x; i < WORDS; i++){
        c.d[i] = a.d[i-x];
    }
    return c;
}

BigNum LongShiftDigitsToLow(const BigNum& a, int x){
    BigNum c = {};
    if (x >= WORDS) return c;
    for(int i = 0; i < WORDS-x; i++){
        c.d[i] = a.d[i+x];
    }
```

```
    return c;
}

BigNum LongShiftBitsToHigh(const BigNum& a, int x){
    int digits = x / W;
    int bits = x % W;
    BigNum c = LongShiftDigitsToHigh(a, digits);
    uint32_t tail = 0;
    uint64_t temp =0;
    for(int i =0; i <WORDS; i++){
        temp = (uint64_t)c.d[i] <<bits;
        c.d[i] = (temp & MASK) | tail;
        tail = temp >> W;
    }
    return c;
}

BigNum LongShiftBitsToLow(const BigNum& a, int x){
    int digits = x / W;
    int bits = x % W;
    BigNum c = LongShiftDigitsToLow(a, digits);
    uint32_t tail = 0;
    uint64_t temp =0;
    for(int i =WORDS-1; i >=0; i --){
        temp = ((uint64_t)c.d[i]<<W) >>bits;
        c.d[i] = (temp >> W) | tail;
        tail = temp & MASK;
    }
    return c;
}

void LongMul(const BigNum& a, const BigNum& b, BigNum& c){
    BigNum tempC = {};
    BigNum temp = {};
    for(int i =0; i < WORDS; i++){
        if (b.d[i] == 0) continue;
        LongMulOneDigit(a,b.d[i],temp);
        LongAdd(tempC, LongShiftDigitsToHigh(temp,i), tempC);
    }
    c = tempC;
}

void LongSquare(const BigNum& a, BigNum& b) {
    BigNum t = {};
    BigNum d = {};
    uint32_t carry;
    uint64_t temp = 0;
    int n = (BitLength(a) + W - 1) / W;

    for(int i =0; i <n; i++){
        if (a.d[i] == 0) continue;
        carry=0;
        for(int j = i + 1; j < n && i + j < WORDS; j++){
            temp = (uint64_t)t.d[i + j] + (uint64_t)a.d[i] * a.d[j] + carry;
            t.d[i + j] = temp & MASK;
            carry = temp >> W;
        }
        if (i + n < WORDS) t.d[i + n] = carry;
    }

```

```
    t = LongShiftBitsToHigh(t, 1);

    for(int i = 0; i < n && 2 * i < WORDS; i++){
        temp = (uint64_t)a.d[i] * a.d[i];
        d.d[2 * i] = temp & MASK;
        d.d[2 * i + 1] = temp >> W;
    }

    LongAdd(t, d, b);
}

int BitLength(const BigNum& a){

    for(int i = WORDS-1; i >=0; i--){
        if(a.d[i]!=0){
            uint32_t v = a.d[i];
            int bits = 0;
            while (v!=0){
                v = v>>1;
                bits++;
            }
            return i*W +bits;
        }
    }
    return 0;
}

bool LongDivMod(const BigNum& a,const BigNum& b,BigNum& q,BigNum& r){
    int k = BitLength(b);
    if (k ==0){
        q = {};
        r = {};
        return false;
    }
    int t ;
    int s;
    BigNum c;
    BigNum tempR = a;
    BigNum tempQ = {};
    while(LongCmp(tempR,b)!=-1){
        t = BitLength(tempR);
        c = LongShiftBitsToHigh(b, t-k);
        if (LongCmp(c,tempR)==1){
            t--;
            c = LongShiftBitsToHigh(b,t-k);
        }
        LongSub(tempR,c,tempR);
        s = t-k;
        tempQ.d[s/W] |= 1u << (s%W);
    }
    q = tempQ;
    r = tempR;
    return true;
}

void LongPower(const BigNum& a,const BigNum& b,BigNum& c){
    BigNum tempC = {};
    tempC.d[0] = 1;
    for(int i = BitLength(b)-1; i >=0; i--){
        if ((b.d[i / W] >> (i % W)) & 1){
```

```
            LongMul(tempC, a, tempC);
        }
        if(i!=0){
            LongSquare(tempC,tempC);
        }
    }
    c = tempC;
}

static int CharValue(char ch, int bitsPerChar){
    int v = -1;
    if (ch >= '0' && ch <= '9') v = ch - '0';
    else if (ch >= 'a' && ch <= 'f') v = ch - 'a' + 10;
    else if (ch >= 'A' && ch <= 'F') v = ch - 'A' + 10;
    if (v >= (1 << bitsPerChar)) return -1;
    return v;
}

static bool FromPow2String(const std::string& s, int bitsPerChar, BigNum& a){
    if (s.empty()) return false;
    BigNum t = {};
    int pos = 0;
    for (size_t k = s.size(); k > 0; k--){
        int v = CharValue(s[k - 1], bitsPerChar);
        if (v < 0) return false;
        if (pos >= BITS){
            if (v != 0) return false;
            continue;
        }
        t.d[pos / W] |= (uint32_t)v << (pos % W);
        pos += bitsPerChar;
    }
    a = t;
    return true;
}

static std::string ToPow2String(const BigNum& a, int bitsPerChar){
    const char* digits = "0123456789ABCDEF";
    int n = BitLength(a);
    if (n == 0) return "0";
    int chars = (n + bitsPerChar - 1) / bitsPerChar;
    uint32_t mask = (1u << bitsPerChar) - 1;
    std::string s;
    for (int pos = (chars - 1) * bitsPerChar; pos >= 0; pos -= bitsPerChar){
        s += digits[(a.d[pos / W] >> (pos % W)) & mask];
    }
    return s;
}

bool FromHex(const std::string& s, BigNum& a){
    if (s.size() >= 2 && s[0] == '0' && (s[1] == 'x' || s[1] == 'X'))
        return FromPow2String(s.substr(2), 4, a);
    return FromPow2String(s, 4, a);
}

std::string ToHex(const BigNum& a){
    return ToPow2String(a, 4);
}

bool FromBin(const std::string& s, BigNum& a){
```

```
256     if (s.size() >= 2 && s[0] == '0' && (s[1] == 'b' || s[1] == 'B'))
257         return FromPow2String(s.substr(2), 1, a);
258     return FromPow2String(s, 1, a);
259 }
260 
261 std::string ToBin(const BigNum& a){
262     return ToPow2String(a, 1);
263 }
```

6.3 Файл tests.h

```
1 #pragma once
2 
3 int runTests();
```

6.4 Файл tests.cpp

```
1  #include "tests.h"
2  #include "bignumarifmetic.h"
3  #include <cstdio>
4  #include <random>
5  #include <string>
6  
7  static std::mt19937 rng(20261009);
8  
9  static BigNum RandomBigNum(int maxBits = BITS){
10     int bits = 1 + rng() % maxBits;
11     BigNum x = {};
12     for (int i = 0; i < WORDS && i * W < bits; i++) x.d[i] = rng();
13     if (bits % W != 0) x.d[bits / W] &= (1u << (bits % W)) - 1;
14     return x;
15 }
16 
17 static BigNum Add(const BigNum& a, const BigNum& b){ BigNum c; LongAdd(a, b, c); return c
       ; }
18 static BigNum Sub(const BigNum& a, const BigNum& b){ BigNum c; LongSub(a, b, c); return c
       ; }
19 static BigNum Mul(const BigNum& a, const BigNum& b){ BigNum c; LongMul(a, b, c); return c
       ; }
20 static BigNum Sqr(const BigNum& a){ BigNum c; LongSquare(a, c); return c; }
21 static BigNum Pow(const BigNum& a, const BigNum& b){ BigNum c; LongPower(a, b, c); return
        c; }
22 static BigNum Hex(const std::string& s){ BigNum a; FromHex(s, a); return a; }
23 static bool   Eq(const BigNum& a, const BigNum& b){ return LongCmp(a, b) == 0; }
24 
25 static void Show(const char* name, const BigNum& x){
26     printf("  %s = %s\n", name, ToHex(x).c_str());
27 }
28 
29 static int failed = 0;
30 
31 static void Report(const char* name, bool ok){
32     printf("%s: %s\n", name, ok ? "Passed!" : "Failed!");
33     if (!ok) failed++;
34 }
35 
36 
```

```
template <class F>
static bool ForRandom(int count, F check){
    for (int t = 0; t < count; t++)
        if (!check()) return false;
    return true;
}

const int RANDOM = 1000;

int runTests(){
    printf("--- Running Tests (%d bit, all operations mod 2^%d) ---\n\n", BITS, BITS);
    failed = 0;


    BigNum a = Hex("123456789ABCDEF0FEDCBA9876543210");
    BigNum b = Hex("0F1E2D3C4B5A69788796A5B4C3D2E1F0");
    BigNum c = Hex("1234567890ABCDEF");
    Show("a", a); Show("b", b); Show("c", c);
    printf("\n");

    Show("(a+b)*c  ", Mul(Add(a, b), c));
    Show("a*c + b*c", Add(Mul(a, c), Mul(b, c)));
    Report("Test 1 [(a+b)*c == c*(a+b) == a*c + b*c]",
           ForRandom(RANDOM, []{
               BigNum x = RandomBigNum(), y = RandomBigNum(), z = RandomBigNum();
               BigNum l = Mul(Add(x, y), z);
               return Eq(l, Mul(z, Add(x, y))) && Eq(l, Add(Mul(x, z), Mul(y, z)));
           }));

    BigNum sum = {};
    for (int k = 0; k < 100; k++) sum = Add(sum, a);
    BigNum na; LongMulOneDigit(a, 100, na);
    Show("100*a      ", na);
    Show("a + ... + a", sum);
    Report("Test 2 [n*a == a+a+...+a, n = 100..1000]",
           Eq(na, sum) && ForRandom(RANDOM, []{
               BigNum x = RandomBigNum(), s = {}, m;
               uint32_t n = 100 + rng() % 901;
               for (uint32_t k = 0; k < n; k++) s = Add(s, x);
               LongMulOneDigit(x, n, m);
               return Eq(m, s) && Eq(Mul(ToBigNum(n), x), s);
           }));

    Show("a + b", Add(a, b)); Show("a - b", Sub(a, b));
    BigNum max = Hex(std::string(BITS / 4, 'F')), r;
    uint32_t carry = LongAdd(max, ToBigNum(1), r);
    bool overflow = Eq(r, ToBigNum(0)) && carry == 1;
    uint32_t borrow = LongSub(ToBigNum(0), ToBigNum(1), r);
    bool underflow = Eq(r, max) && borrow == 1;
    Report("Test 3 [a+b == b+a, (a+b)-b == a, a-a == 0, carry/borrow]",
           Eq(Add(a, b), Hex("215283B4E61748698673604D3A271400")) &&
           Eq(Sub(a, b), Hex("316293C4F627578774614E3B2815020")) && overflow && underflow
     &&
           ForRandom(RANDOM, []{
               BigNum x = RandomBigNum(), y = RandomBigNum();
               return Eq(Add(x, y), Add(y, x)) && Eq(Sub(Add(x, y), y), x) && Eq(Sub(x, x
    ), ToBigNum(0));
           }));

    Show("a * b", Mul(a, b)); Show("a^2  ", Sqr(a));
```

```
95      Report("Test 4 [a*b == b*a, a^2 == a*a, (a+b)^2 == a^2 + 2ab + b^2]",
96             Eq(Mul(a, b), Hex("113366
        AB0066DE57C0175D92B6C9CBBB8A47F4901A93FD55EF9A562300EFF00")) &&
97             Eq(Sqr(a), Hex("14
        B66DC33F6ACDCCA2148A6A1A009454495D294750DF8CCDEEC6CD7A44A4100")) &&
98             Eq(Sqr(ToBigNum(0xFFFFFFFF)), Hex("FFFFFFFE00000001")) &&
99             ForRandom(RANDOM, []{
100                BigNum x = RandomBigNum(), y = RandomBigNum();
101                BigNum two_xy = LongShiftBitsToHigh(Mul(x, y), 1);
102                return Eq(Mul(x, y), Mul(y, x)) && Eq(Sqr(x), Mul(x, x)) &&
103                       Eq(Sqr(Add(x, y)), Add(Add(Sqr(x), two_xy), Sqr(y)));
104            }));
105
106     BigNum a2 = Add(Sqr(a), ToBigNum(12345)), q;
107     LongDivMod(a2, b, q, r);
108     Show("a^2 + 12345", a2); Show("q = (a^2 + 12345) / b", q); Show("r = (a^2 + 12345) %
        b", r);
109     Report("Test 5 [a == b*q + r, 0 <= r < b, div by 0 -> error]",
110            Eq(q, Hex("15EBD2CAD3EE192EC2C99D3DAAE4FCFD")) && Eq(r, Hex("61
        F7E44949059EAB1A44063ECB8E709")) &&
111            !LongDivMod(a, ToBigNum(0), q, r) &&
112            ForRandom(RANDOM, []{
113                BigNum x = RandomBigNum(), y, qq, rr;
114                do y = RandomBigNum(); while (BitLength(y) == 0);
115                return LongDivMod(x, y, qq, rr) && Eq(Add(Mul(y, qq), rr), x) && LongCmp(
        rr, y) < 0;
116            }));
117
118     Show("3^200", Pow(ToBigNum(3), ToBigNum(200)));
119     Report("Test 6 [a^(m+n) == a^m * a^n, a^(m*n) == (a^m)^n, a^0 == 1, a^1 == a]",
120            Eq(Pow(ToBigNum(3), ToBigNum(200)),
121               Hex("1
        FD5863C3EB0469EC21A937A76F3432FFD73D97E447606B683ECF6F6E4A7AE225BFAFF1EAAF8B0A1")) &&
122            Eq(Pow(a, ToBigNum(0)), ToBigNum(1)) && Eq(Pow(a, ToBigNum(1)), a) &&
123            Eq(Pow(ToBigNum(2), ToBigNum(BITS)), ToBigNum(0)) &&
124            ForRandom(100, []{
125                BigNum x = RandomBigNum();
126                uint32_t m = rng() % 100000, n = rng() % 100000;
127                BigNum xm = Pow(x, ToBigNum(m));
128                return Eq(Pow(x, ToBigNum((uint64_t)m + n)), Mul(xm, Pow(x, ToBigNum(n))))
        &&
129                       Eq(Pow(x, ToBigNum((uint64_t)m * n)), Pow(xm, ToBigNum(n)));
130            }));
131
132     Show("a << 100", LongShiftBitsToHigh(a, 100)); Show("a >> 8  ", LongShiftBitsToLow(a,
        8));
133     Report("Test 7 [a << k == a*2^k, a >> k == a / 2^k, BitLength(2^k) == k+1]",
134            BitLength(ToBigNum(0)) == 0 && BitLength(ToBigNum(1)) == 1 && BitLength(a) ==
        125 &&
135            ForRandom(300, []{
136                int k = rng() % BITS;
137                BigNum x = RandomBigNum(), p = Pow(ToBigNum(2), ToBigNum(k)), qq, rr;
138                LongDivMod(x, p, qq, rr);
139                return Eq(LongShiftBitsToHigh(x, k), Mul(x, p)) && Eq(LongShiftBitsToLow(x
        , k), qq) &&
140                       BitLength(p) == k + 1;
141            }));
142
143     printf("  ToBin(c) = %s\n", ToBin(c).c_str());
144     BigNum t;
```

```
145     Report("Test 8 [FromHex(ToHex(a)) == a, FromBin(ToBin(a)) == a, bad strings rejected]
        ",
146             FromHex("0x1f", t) && Eq(t, ToBigNum(31)) && ToHex(ToBigNum(0)) == "0" &&
147             FromBin("0b101", t) && Eq(t, ToBigNum(5)) &&
148             !FromHex("12G4", t) && !FromBin("102", t) && !FromHex("", t) &&
149             !FromHex("1" + std::string(BITS / 4, '0'), t) &&
150             ForRandom(RANDOM, []{
151                 BigNum x = RandomBigNum(), y;
152                 return FromHex(ToHex(x), y) && Eq(x, y) && FromBin(ToBin(x), y) && Eq(x, y
        );
153             }));
154 
155     printf("\n%s (failed: %d)\n----------------------------\n\n",
156             failed ? "SOME TESTS FAILED" : "All tests passed", failed);
157     return failed;
158 }
```

6.5 Файл benchmarks.h

```
1 #pragma once
2 
3 void runBenchmarks();
```

6.6 Файл benchmarks.cpp

```
1  #include "benchmarks.h"
2  #include "bignumarifmetic.h"
3  #include <cstdio>
4  #include <chrono>
5  #include <random>
6  #include <string>
7  #include <intrin.h>
8  
9  static std::mt19937 rng(20261012);
10 
11 static BigNum RandomBigNum(int bits){
12     BigNum x = {};
13     for (int i = 0; i < WORDS && i * W < bits; i++) x.d[i] = rng();
14     if (bits % W != 0) x.d[bits / W] &= (1u << (bits % W)) - 1;
15     return x;
16 }
17 
18 const int POOL = 1024;
19 const int P = POOL - 1;
20 const int D = WORDS - 1;
21 static BigNum A[POOL], B[POOL], Half[POOL], Small[POOL], E64[POOL];
22 static BigNum A128[POOL], B128[POOL], A1024[POOL], B1024[POOL];
23 static int K[POOL];
24 static std::string HexStr[POOL];
25 
26 static volatile uint32_t sink;
27 
28 template <class F>
29 static void Bench(const char* name, int iterations, F op){
30     for (int i = 0; i < iterations / 10 + 1; i++) op(i);
31     auto start = std::chrono::steady_clock::now();
32     uint64_t c0 = __rdtsc();
```

```
    for (int i = 0; i < iterations; i++) op(i);
    uint64_t c1 = __rdtsc();
    auto end = std::chrono::steady_clock::now();
    double ns = std::chrono::duration<double, std::nano>(end - start).count() / iterations;
    printf("| %-30s | %10d | %12.1f | %12.0f |\n", name, iterations, ns, (double)(c1 - c0) / iterations);
}

void runBenchmarks(){
    printf("--- Running Benchmarks (%d bit, %d random numbers per operand) ---\n\n", BITS, POOL);

    for (int i = 0; i < POOL; i++){
        A[i] = RandomBigNum(BITS);
        B[i] = RandomBigNum(BITS);
        Half[i] = RandomBigNum(BITS / 2);  Half[i].d[WORDS / 2 - 1] |= 1u << 31;
        Small[i] = RandomBigNum(W);        Small[i].d[0] |= 1;
        E64[i] = RandomBigNum(64);
        K[i] = rng() % BITS;
        HexStr[i] = ToHex(A[i]);
        A128[i] = RandomBigNum(128);       B128[i] = RandomBigNum(128);
        A1024[i] = RandomBigNum(1024);     B1024[i] = RandomBigNum(1024);
    }

    printf("| %-30s | %10s | %12s | %12s |\n", "Operation", "Iterations", "ns / op", "cycles / op");
    printf("|--------------------------------|------------|--------------|--------------|\n");

    Bench("LongAdd", 1000000, [](int i){
        BigNum c; LongAdd(A[i & P], B[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongSub", 1000000, [](int i){
        BigNum c; LongSub(A[i & P], B[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongCmp", 1000000, [](int i){
        sink = sink ^ (uint32_t)LongCmp(A[i & P], B[i & P]); });
    Bench("BitLength", 1000000, [](int i){
        sink = sink ^ (uint32_t)BitLength(A[i & P]); });
    Bench("LongMulOneDigit", 1000000, [](int i){
        BigNum c; LongMulOneDigit(A[i & P], B[i & P].d[0], c); sink = sink ^ c.d[i & D]; });
    Bench("LongShiftBitsToHigh", 1000000, [](int i){
        sink = sink ^ LongShiftBitsToHigh(A[i & P], K[i & P]).d[i & D]; });
    Bench("LongShiftBitsToLow", 1000000, [](int i){
        sink = sink ^ LongShiftBitsToLow(A[i & P], K[i & P]).d[i & D]; });
    Bench("LongMul (128 bit)", 200000, [](int i){
        BigNum c; LongMul(A128[i & P], B128[i & P], c); sink = sink ^ c.d[i & 7]; });
    Bench("LongMul (1024 bit)", 50000, [](int i){
        BigNum c; LongMul(A1024[i & P], B1024[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongMul (2048 bit)", 50000, [](int i){
        BigNum c; LongMul(A[i & P], B[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongSquare (128 bit)", 1000000, [](int i){
        BigNum c; LongSquare(A128[i & P], c); sink = sink ^ c.d[i & 7]; });
    Bench("LongSquare (1024 bit)", 200000, [](int i){
        BigNum c; LongSquare(A1024[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongSquare (2048 bit)", 200000, [](int i){
        BigNum c; LongSquare(A[i & P], c); sink = sink ^ c.d[i & D]; });
    Bench("LongDivMod (2048 / 1024 bit)", 2000, [](int i){
        BigNum q, r; LongDivMod(A[i & P], Half[i & P], q, r); sink = sink ^ q.d[i & 31] ^ r.d[i & 31]; });
```

```
86      Bench("LongDivMod (2048 / 32 bit)", 1000, [](int i){
87          BigNum q, r; LongDivMod(A[i & P], Small[i & P], q, r); sink = sink ^ q.d[i & D] ^
        r.d[0]; });
88      Bench("LongPower (exponent 64 bit)", 1000, [](int i){
89          BigNum c; LongPower(A[i & P], E64[i & P], c); sink = sink ^ c.d[i & D]; });
90      Bench("LongPower (exponent 2048 bit)", 30, [](int i){
91          BigNum c; LongPower(A[i & P], B[i & P], c); sink = sink ^ c.d[i & D]; });
92      Bench("ToHex", 200000, [](int i){
93          std::string s = ToHex(A[i & P]); sink = sink ^ (uint32_t)s[i % s.size()]; });
94      Bench("FromHex", 200000, [](int i){
95          BigNum c; FromHex(HexStr[i & P], c); sink = sink ^ c.d[i & D]; });
96
97      printf("\ncycles = TSC (__rdtsc), counts at the nominal CPU frequency\n");
98      printf("-------------------------------\n\n");
99  }
```

6.7 Файл main.cpp

```
1  #include "tests.h"
2  #include "benchmarks.h"
3  #define NOMINMAX
4  #include <windows.h>
5  #include <cstdio>
6
7  int main(){
8      SetConsoleOutputCP(CP_UTF8);
9      int failed = runTests();
10     runBenchmarks();
11     printf("asdads\n");
12     return failed;
13 }
```